

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ)**

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №4

по курсу «Технологии параллельного программирования»

Работа с матрицами. Метод Гаусса

Выполнил: Ахметшин Б.Р.

Группа: М8О-403Б-22

Преподаватели: А.Ю. Морозов,

Е.Е. Заяц

Москва, 2026

Условие

Цель работы: реализовать метод Гаусса на GPU с выбором главного элемента по столбцу, используя библиотеку Thrust для поиска максимального элемента, и провести исследование производительности с помощью профилировщика `nvprof`.

Выполненная реализация соответствует варианту 3 (решение квадратной СЛАУ $Ax = b$).

Программное и аппаратное обеспечение

GPU

Имя	NVIDIA Tesla T4
Compute Capability	7.5
Объем глобальной памяти	15637086208
Разделяемая память на блок	49152
Константная память	65536
Регистров на блок	65536
Макс. потоков на блок	(1024, 1024, 64)
Макс. размер сетки	(2147483647, 65535, 65535)
Количество мультипроцессоров	40

CPU

Архитектура	x86_64
CPU op-mode(s)	32-bit, 64-bit
Address sizes	48 bits physical, 48 bits virtual
Порядок байт	Little Endian
CPU(s)	12
Имя модели	AMD Ryzen 5 7600 6-Core Processor
Ядер / потоков	6 / 12

RAM

64 GB

ОС

Ubuntu 24.04.4 LTS

Компилятор

nvcc, g++

Метод решения

Используется классический метод Гаусса с частичным выбором главного элемента по столбцу:

1. на шаге k выбирается строка с максимальным по модулю элементом в текущем столбце (`thrust::max_element`);
2. выполняется перестановка строк;
3. вычисляются коэффициенты исключения;
4. обнуляются элементы ниже главного в двумерной CUDA-сетке потоков;
5. после прямого хода выполняется обратный ход (back substitution).

Описание программы

- `lab4.cu` — GPU-реализация: kernels для перестановки строк, вычисления коэффициентов и исключения; поиск pivot через Thrust.
- `lab4-cpu.cpp` — CPU-реализация того же алгоритма для сравнения.
- `generate_case.py` — генератор тестовых систем (диагонально доминирующих).
- `run_case.sh` — запуск кейса с файла.
- `profile_nvprof.sh` — запуск с профилировщиком nvprof.

Внутренние замеры:

- GPU: `kernel_ms`, `total_ms` (`timings_gpu_lab4.txt`);

- CPU: `total_ms (timings_cpu_lab4.txt)`;
- параметры блоков GPU можно задавать через `LAB4_SWAP_THREADS`, `LAB4_FACTOR_THREADS`, `LAB4_ELIM_BLOCK_X`, `LAB4_ELIM_BLOCK_Y`.

Как из кастомных параметров получается `launch`-конфигурация `kernel`'ов:

- `swapRowsKernel`:
`block = (LAB4_SWAP_THREADS, 1, 1),`
`grid.x = ceil((n+1)/LAB4_SWAP_THREADS).`
- `computeFactorsKernel`:
`block = (LAB4_FACTOR_THREADS, 1, 1),`
`grid.x = ceil((n-(k+1))/LAB4_FACTOR_THREADS).`
- `eliminateKernel`:
`block = (LAB4_ELIM_BLOCK_X, LAB4_ELIM_BLOCK_Y, 1),`
`grid.x = ceil((n+1-k)/LAB4_ELIM_BLOCK_X),`
`grid.y = ceil((n-(k+1))/LAB4_ELIM_BLOCK_Y).`
- Поиск главного элемента через `thrust::max_element` запускает внутренние `kernel`'ы Thrust/CUB; их конфигурация в `nvprof` отображается отдельно и напрямую через `LAB4_*` не задается.

Результаты

Замеры времени:

No	n	CUDA config	CPU, ms	GPU kernel, ms	GPU total, ms
1	128	swap=128, factor=128, elim=(16, 16)	0.218658	794.482422	1093.930854
2	128	swap=256, factor=256, elim=(32, 32)	0.218658	64.523422	271.532119
3	128	swap=512, factor=512, elim=(64, 64)	0.218658	99.555840	296.643681
4	512	swap=128,	13.721943	599.301819	795.664067

		factor=128,				
		elim=(16, 16)				
+---	+---	+-----+	+-----+	+-----+	+-----+	+---
5	512	swap=256,	13.721943	514.086426	702.878280	
		factor=256,				
		elim=(32, 32)				
+---	+---	+-----+	+-----+	+-----+	+-----+	+---
6	512	swap=512,	13.721943	570.717224	780.682874	
		factor=512,				
		elim=(64, 64)				
+---	+---	+-----+	+-----+	+-----+	+-----+	+---
7	2048	swap=128,	850.001810	2534.226562	2914.033293	
		factor=128,				
		elim=(16, 16)				
+---	+---	+-----+	+-----+	+-----+	+-----+	+---
8	2048	swap=256,	850.001810	2599.576904	2891.329117	
		factor=256,				
		elim=(32, 32)				
+---	+---	+-----+	+-----+	+-----+	+-----+	+---
9	2048	swap=512,	850.001810	2580.171143	2901.593170	
		factor=512,				
		elim=(64, 64)				
+---	+---	+-----+	+-----+	+-----+	+-----+	+---

Логи профилировщика nvprof (сводка):

```
==10411== NVPROF is profiling process 10411, command: ./lab4
==10411== Profiling application: ./lab4
==10411== Profiling result:
   Type  Time(%)      Time   Calls    Avg      Min      Max   Name
GPU activities:  36.99%  785.59us    128  6.1370us  4.1600us  7.5520us  [thrust::
  cuda_cub reduce arg_max kernel]
                22.05%  468.38us    127  3.6880us  3.4560us  4.3840us
                  computeFactorsKernel(double const *, double*, int, int, int)
                20.24%  429.91us    257  1.6720us  1.5680us  12.192us  [CUDA memcpy
                  DtoH]
                20.08%  426.55us    127  3.3580us  2.8800us  4.4480us  eliminateKernel
                  (double*, double const *, int, int, int)
                 0.64%  13.536us      1  13.536us  13.536us  13.536us  [CUDA memcpy
                  HtoD]
```

Анализ профилировщика:

- Для профиля $n = 128$ (параметры: $\text{swap}/\text{factor} = 256$, $\text{elim} = (16,16)$): основной вклад дают три группы GPU-активностей: `thrust reduce` (36.99%), `computeFactorsKernel` (22.05%) и `eliminateKernel` (20.08%). Также заметна доля частых DtoH копирований малых объемов (20.24%).
- В профиле видно большое число очень коротких запусков (порядка $3-7 \mu\text{s}$ на kernel). Для малого размера матрицы это приводит к доминированию накладных расходов запуска/синхронизаций.
- Копирование крупных массивов не является узким местом: HtoD 129 KB занимает малую долю (0.64%), тогда как повторяющиеся мелкие DtoH добавляют значимую задержку.
- По таблице замеров видно, что рост n увеличивает долю полезной работы ядра (например, для $n = 2048$ $\text{kernel_ms} \approx 2.58-2.60$ с при $\text{total_ms} \approx 2.89$ с), однако реализация все равно уступает CPU по общему времени.

Выводы

По результатам экспериментов и профилирования получены следующие выводы:

- Для всех исследованных размеров ($n = 128, 512, 2048$) текущая GPU-реализация оказалась медленнее CPU по полному времени выполнения. Наиболее заметное отставание наблюдается на малых размерах, где накладные расходы доминируют.
- На $n = 128$ лучшая из протестированных конфигураций (swap=256, factor=256, elim=(32,32)) дает $GPU\ total = 271.53\ ms$ против $CPU = 0.22\ ms$.
- На $n = 512$ лучшая конфигурация (swap=256, factor=256, elim=(32,32)) дает $GPU\ total = 702.88\ ms$ против $CPU = 13.72\ ms$.
- На $n = 2048$ лучшая конфигурация (swap=256, factor=256, elim=(32,32)) дает $GPU\ total = 2891.33\ ms$ против $CPU = 850.00\ ms$.
- Профиль **nvprof** подтверждает, что существенную долю времени занимают многократные короткие запуски kernel'ов и частые мелкие синхронизирующие копирования DtoH. Это объясняет, почему даже при корректной двумерной сетке потоков GPU не показывает ускорения в текущей постановке.
- Для повышения эффективности целесообразно уменьшать число запусков kernel'ов и CPU-GPU синхронизаций (укрупнение шагов, сокращение мелких DtoH, перенос большего объема логики выбора pivot на GPU).